Small Multiples & Faceted Displays

Workshop 3 — Module 8

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

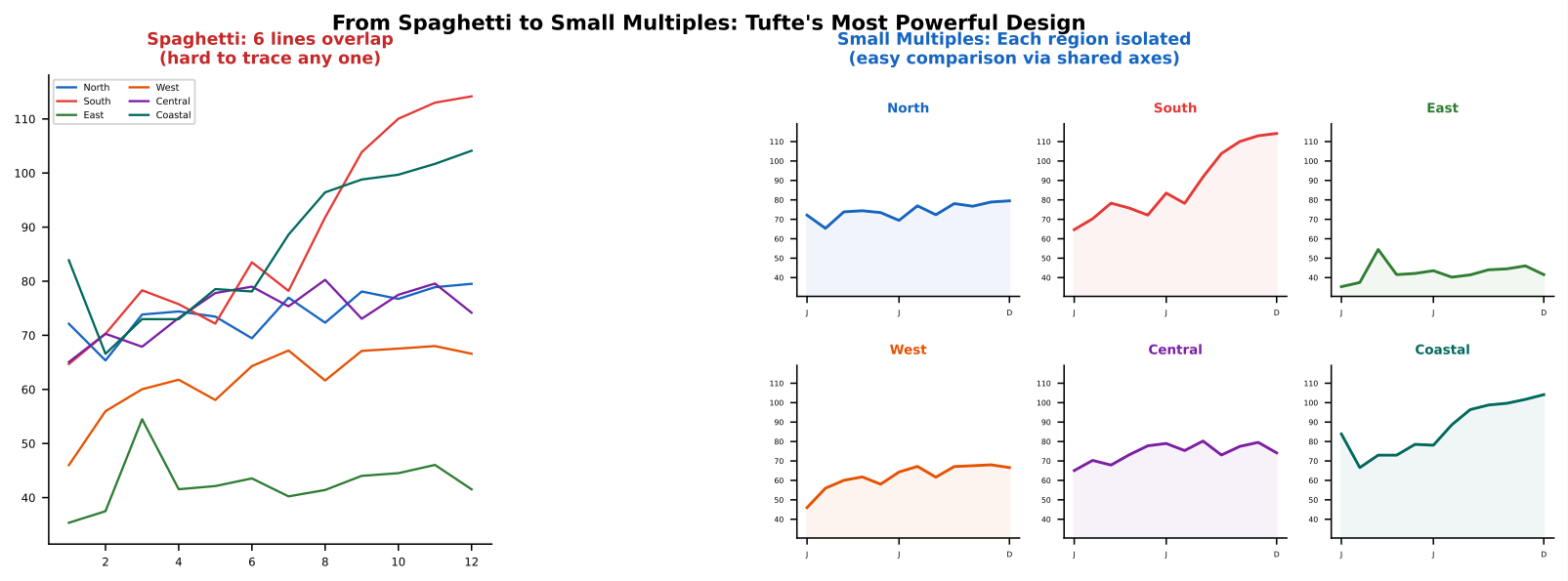
Module Roadmap

1. Tufte's small multiples revisited as a chart type
2. Cleveland's Trellis philosophy: four principles
3. Composition in R: patchwork operators and cowplot
4. Composition in Python: GridSpec layout patterns
5. Facet (homogeneous) vs patchwork (heterogeneous) decision
6. Building multi-panel dashboards as static figures

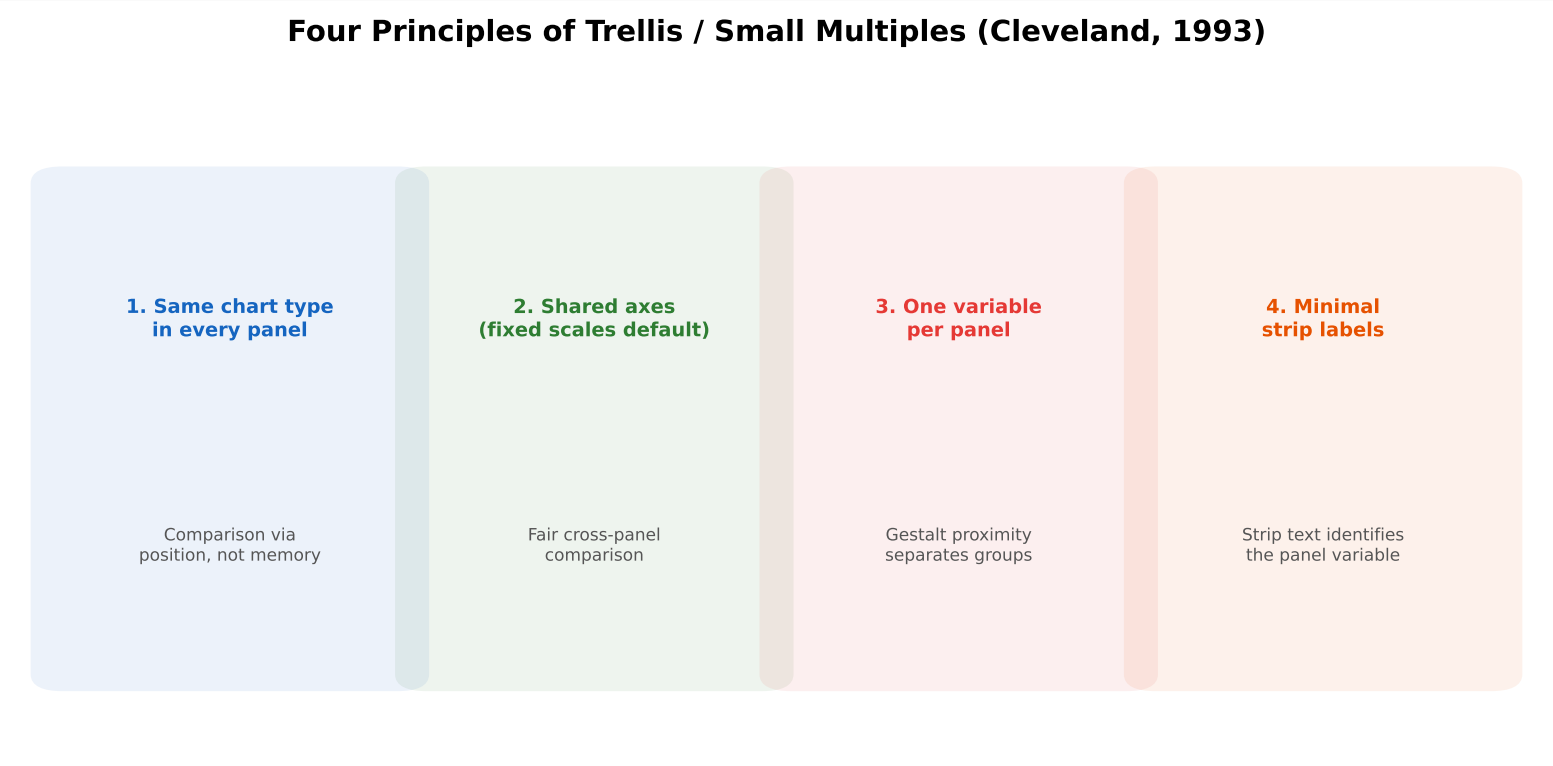
Learning Outcome
You will compose heterogeneous multi-panel figures using `patchwork` (R) and `GridSpec` (Python), choosing between faceting and manual composition based on panel homogeneity.

From Spaghetti to Small Multiples

The Motivation: 6 Lines $\rightarrow$ 6 Panels



Trellis Philosophy: Four Principles



Pro-Tip

Cleveland (1993) formalised small multiples as **Trellis displays**: same chart type, same axes, one variable per panel. This exploits the pre-attentive system: the reader scans panels by position (Gestalt proximity from W01-M03) rather than by colour discrimination.

Composition in R: patchwork

patchwork Operators

patchwork Operators: The Algebra of Plot Composition

Code	Effect	Use Case
`p1 | p2`	Side by side	Two plots on one row
`p1 / p2`	Stacked	Two plots in a column
`(p1 | p2) / p3`	Nested layout	Top row: 2 plots; bottom row: 1 wide
`(p1 | p2 | p3) / (p4 | p5 | p6)`	Dashboard grid	2 × 3 grid
`+ plot_annotation(tag_levels='a')`	Auto panel labels	(a), (b), (c), ...
`& theme_minimal()`	Apply to all panels	Single theme call for entire composition

patchwork: Complete Example

```r
library(patchwork)

p_bar <- ggplot(df, aes(x = fct_reorder(cat, n), y = n)) +
  geom_col(fill = "#1565C0") + coord_flip() + theme_minimal()

p_scatter <- ggplot(df, aes(x = reviews, y = rating)) +
  geom_point(alpha = 0.3) + scale_x_log10() + theme_minimal()

p_hist <- ggplot(df, aes(x = rating)) +
  geom_histogram(bins = 30, fill = "#2E7D32") + theme_minimal()

p_box <- ggplot(df, aes(x = type, y = rating)) +
  geom_boxplot(fill = "#E3F2FD") + theme_minimal()

# Compose: | = side by side, / = stack
dashboard <- (p_bar | p_scatter) / (p_hist | p_box) +
  plot_annotation(
    title = "Google Play Store: Visual EDA",
    tag_levels = "a") &       # & applies to ALL panels
  theme_minimal(base_size = 9)

ggsave("dashboard.pdf", dashboard, width = 12, height = 8)
```

patchwork: Advanced Layouts

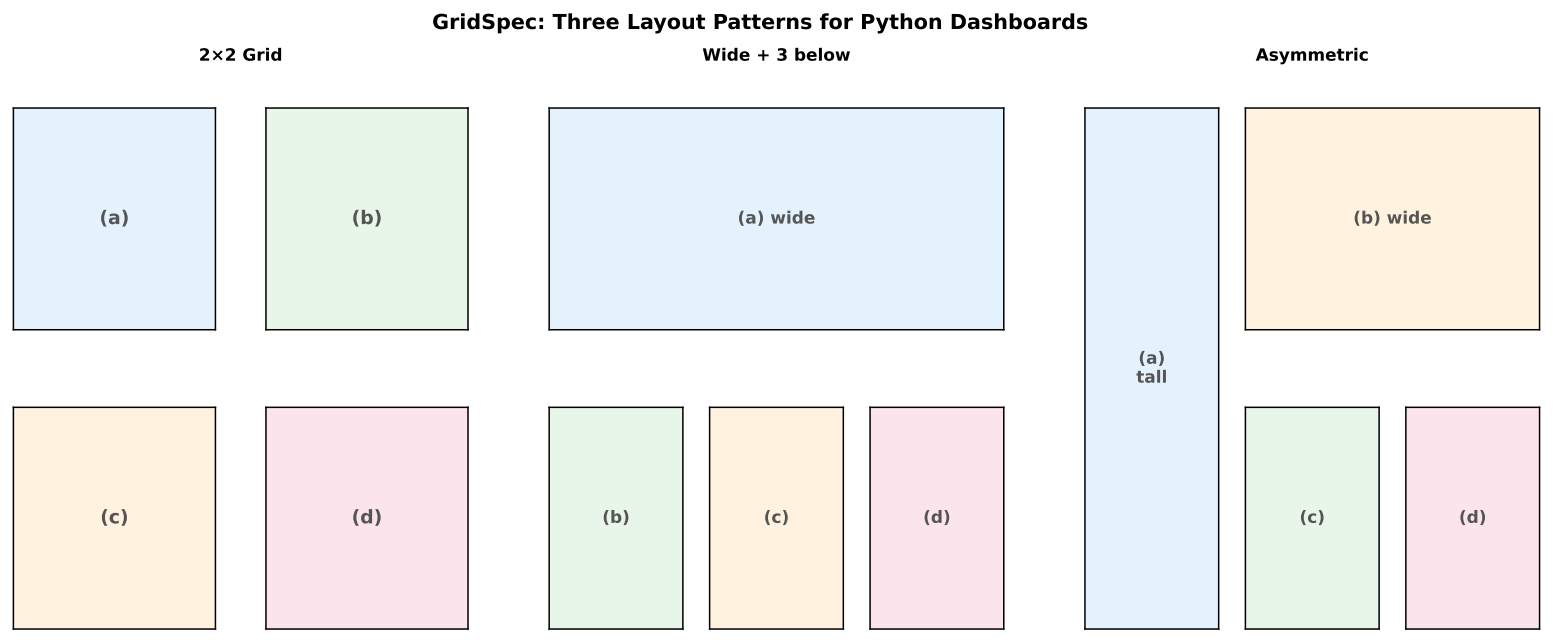
```r
# Unequal widths: bar gets 2x space
(p_bar + p_scatter) + plot_layout(widths = c(2, 1))

# Nested: bar on left, scatter + hist stacked on right
p_bar | (p_scatter / p_hist)

# Insets: small plot inside a large one
p_scatter + inset_element(p_hist, left = 0.6, bottom = 0.6,
                          right = 1, top = 1)

# cowplot alternative (for legacy code)
library(cowplot)
plot_grid(p_bar, p_scatter, p_hist, p_box,
          ncol = 2, labels = "auto",
          rel_widths = c(1.5, 1))
```

Pro-Tip

`patchwork` is the modern standard (Thomas Lin Pedersen, 2020). `cowplot` (Claus Wilke) and `gridExtra` are older alternatives still found in legacy code. Prefer patchwork for new work.

Composition in Python: GridSpec

GridSpec: Three Layout Patterns



GridSpec: Complete Example

```
from matplotlib.gridspec import GridSpec

fig = plt.figure(figsize=(14, 9))
gs = GridSpec(2, 3, figure=fig, hspace=0.4, wspace=0.35)

# Top row: 3 equal panels
ax_bar  = fig.add_subplot(gs[0, 0])     # (a)
ax_hist = fig.add_subplot(gs[0, 1])     # (b)
ax_scat = fig.add_subplot(gs[0, 2])     # (c)

# Bottom row: 1 wide + 2 narrow
ax_line = fig.add_subplot(gs[1, :2])    # (d) spans 2 columns
ax_box  = fig.add_subplot(gs[1, 2])     # (e) single column

# ... plot each panel onto its axes ...

# Global title + caption
fig.suptitle("Dashboard Title", fontsize=13, fontweight="bold")
fig.text(0.5, 0.01, "Source: ...", ha="center", fontsize=8, color="
    #888")

fig.savefig("dashboard.pdf", bbox_inches="tight")
```

subplot_mosaic: Named Layouts (Python 3.4+)

```
# matplotlib 3.4+ : named subplot layout
fig, axes = plt.subplot_mosaic(
    [["bar", "bar", "scatter"],
     ["hist", "box", "scatter"]],
    figsize=(12, 8))

# Access by name instead of index
axes["bar"].barh(cats, vals, color="#1565C0")
axes["scatter"].scatter(x, y, s=10, alpha=0.3)
axes["hist"].hist(ratings, bins=30, color="#2E7D32")
axes["box"].boxplot([free_r, paid_r], labels=["Free","Paid"])
```

Data Insight

`subplot_mosaic` uses an ASCII grid to define layout: repeated names span multiple cells (like `gs[:2]` but more readable). Available since matplotlib 3.4. For older versions, use `GridSpec`.

Facet vs Patchwork

When to Facet vs When to Compose

Facet (homogeneous) vs Patchwork (heterogeneous)

Situation	Tool	Example
Same geom + aes, different data subset	**facet_wrap / facet_grid (automatic)**	Rating histograms per category
Different geom or aes per panel	**patchwork / GridSpec (manual composition)**	Bar + scatter + boxplot + line
Mix of faceted + composed	**patchwork wrapping faceted plots**	Faceted scatter next to a bar chart

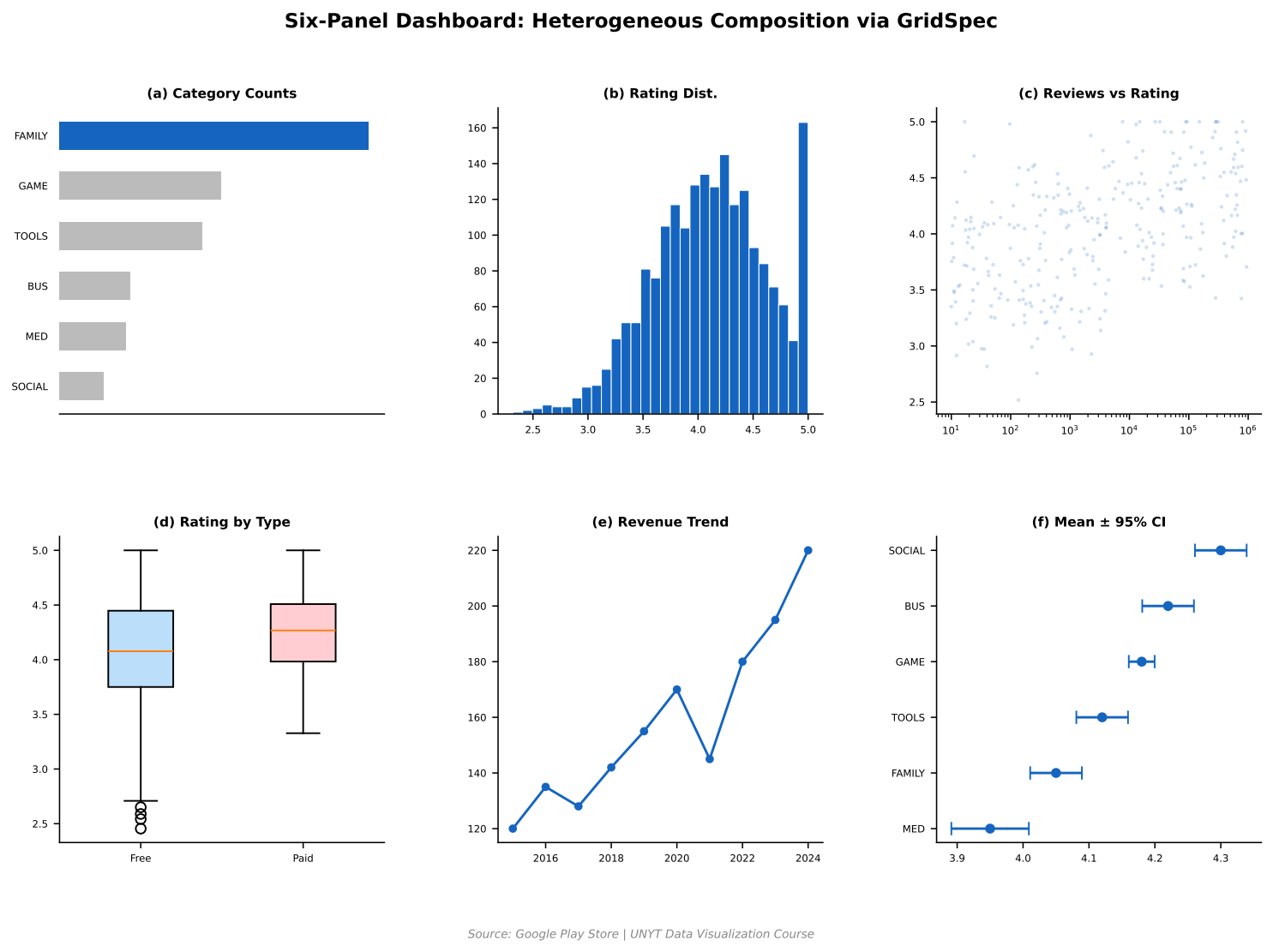
Decision Rule

Question	Answer	Tool
Same geom in all panels?	Yes	`facet_wrap/grid`
Same aes in all panels?	Yes	`facet_wrap/grid`
Different geoms per panel?	Yes	`patchwork / GridSpec`
Need unequal panel sizes?	Yes	`patchwork(widths=)`
Need inset plot?	Yes	`patchwork::inset_element()`
Need panel labels (a,b,c)?	Yes	`tag_levels = "a"`

The simplest rule: if you can write one `ggplot() + facet_*()` call, use faceting. If you need multiple separate `ggplot()` calls with different geoms, use patchwork.

Dashboard Example

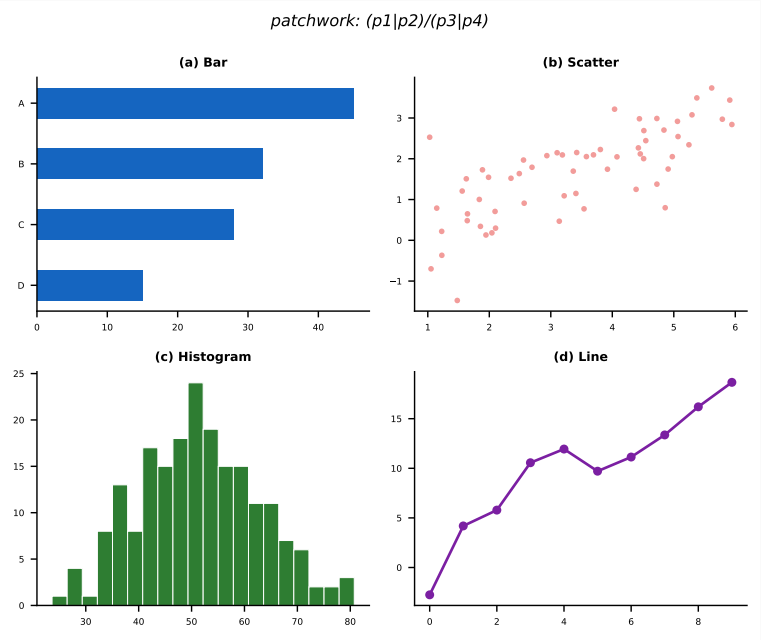
Six-Panel Dashboard via GridSpec



Complete Examples

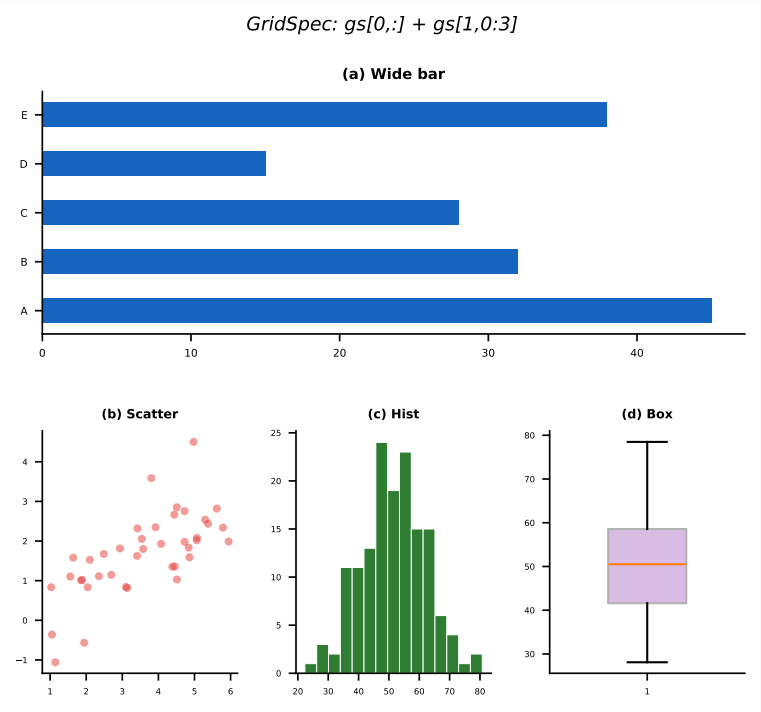
R: patchwork 2×2

```r
library(patchwork)

p1 <- ggplot(df, aes(x=cat,y=n)) +
  geom_col(fill="#1565C0") +
  coord_flip() + theme_minimal() +
  labs(title="(a) Bar")

p2 <- ggplot(df, aes(x=x,y=y)) +
  geom_point(alpha=0.5, size=1,
    color="#E53935") +
  theme_minimal() +
  labs(title="(b) Scatter")

p3 <- ggplot(df, aes(x=val)) +
  geom_histogram(bins=20,
    fill="#2E7D32") +
  theme_minimal() +
  labs(title="(c) Histogram")

p4 <- ggplot(df, aes(x=1:10,y=y)) +
  geom_line(color="#7B1FA2") +
  theme_minimal() +
  labs(title="(d) Line")

(p1 | p2) / (p3 | p4)
```



Python: GridSpec Asymmetric Layout

```python
fig = plt.figure(figsize=(10, 6))
gs = GridSpec(2, 3, figure=fig,
    hspace=0.4, wspace=0.3)

# (a) Top wide bar (spans all 3
    cols)
ax_bar = fig.add_subplot(gs[0, :])
ax_bar.barh(cats, vals,
    color="#1565C0", height=0.5)
ax_bar.set_title("(a) Wide bar")

# (b,c,d) Bottom row: 3 panels
ax_sc = fig.add_subplot(gs[1, 0])
ax_sc.scatter(x, y, s=8,
    c="#E53935", alpha=0.5)
ax_sc.set_title("(b) Scatter")

ax_hi = fig.add_subplot(gs[1, 1])
ax_hi.hist(data, bins=15,
    color="#2E7D32")
ax_hi.set_title("(c) Hist")

ax_bx = fig.add_subplot(gs[1, 2])
ax_bx.boxplot([d1], widths=0.4,
    patch_artist=True)
ax_bx.set_title("(d) Box")
```



Synthesis

Key Takeaways

1. **Small multiples** (Tufte) = same chart, same axes, one variable per panel. Exploits position over colour.
2. **Trellis principles** (Cleveland): same geom, shared axes, one variable per panel, minimal strip labels.
3. **patchwork** (R): `|` = side by side, `/` = stack, `&` = apply to all, `tag_levels` = auto panel labels.
4. **GridSpec** (Python): `gs[row, col]` for placement, `gs[0, :]` for spanning. `subplot_mosaic` for named ASCII layouts.
5. Use **faceting** for homogeneous panels (same geom/aes); **patchwork/GridSpec** for heterogeneous dashboards.
6. Every multi-panel figure needs: **panel labels** (a, b, c), **shared title**, and **source caption**.

Essential Readings

- Cleveland, W. S. (1993). *Visualizing Data*. (Trellis displays.)
- Tufte, E. R. (1983). *The Visual Display of Quantitative Information*, pp. 170–174 (Small multiples.)
- Pedersen, T. L. (2020). patchwork: The Composer of Plots. `https://patchwork.data-imaginist.com`
- Wilke, C. O. (2019). *Fundamentals of Data Visualization*, Chapter 21 (Multi-Panel Figures).

Next Module: Tables as Visualization (W03-M09)